

[5강] RAG Fundamentals

조현수

이화여자대학교 인공지능학과 조교수

저작권 안내

(주)업스테이지가 제공하는 모든 교육 콘텐츠의 지식재산권은
운영 주체인 (주)업스테이지 또는 해당 저작물의 적법한 관리자에게 귀속되어 있습니다.

콘텐츠 일부 또는 전부를 복사, 복제, 판매, 재판매 공개, 공유 등을 할 수 없습니다.

유출될 경우 지식재산권 침해에 대한 책임을 부담할 수 있습니다.

유출에 해당하여 금지되는 행위의 예시는 다음과 같습니다.

- 콘텐츠를 재가공하여 온/오프라인으로 공개하는 행위
- 콘텐츠의 일부 또는 전부를 이용하여 인쇄물을 만드는 행위
- 콘텐츠의 전부 또는 일부를 녹취 또는 녹화하거나 녹취록을 작성하는 행위
- 콘텐츠의 전부 또는 일부를 스크린 캡처하거나 카메라로 촬영하는 행위
- 지인을 포함한 제3자에게 콘텐츠의 일부 또는 전부를 공유하는 행위
- 다른 정보와 결합하여 Upstage Education의 콘텐츠임을 알아볼 수 있는 저작물을 작성, 공개하는 행위
- 제공된 데이터의 일부 혹은 전부를 Upstage Education 프로젝트/실습 수행 이외의 목적으로 사용하는 행위

강의 목표

학습 목표

- Prompt Engineering에서 RAG가 등장한 배경에 대해 이해할 수 있다.
- RAG의 전체적인 파이프라인과 주요 구성 요소에 대해 이해하고 설명할 수 있다.

01. RAG의 배경

- (1) Parametric vs. Non-parametric 지식
- (2) RAG의 목적

02. RAG 파이프라인

- (1) Knowledge Base
- (2) Chunking
- (3) Sparse vs. Dense Retrieval
- (4) Reranking

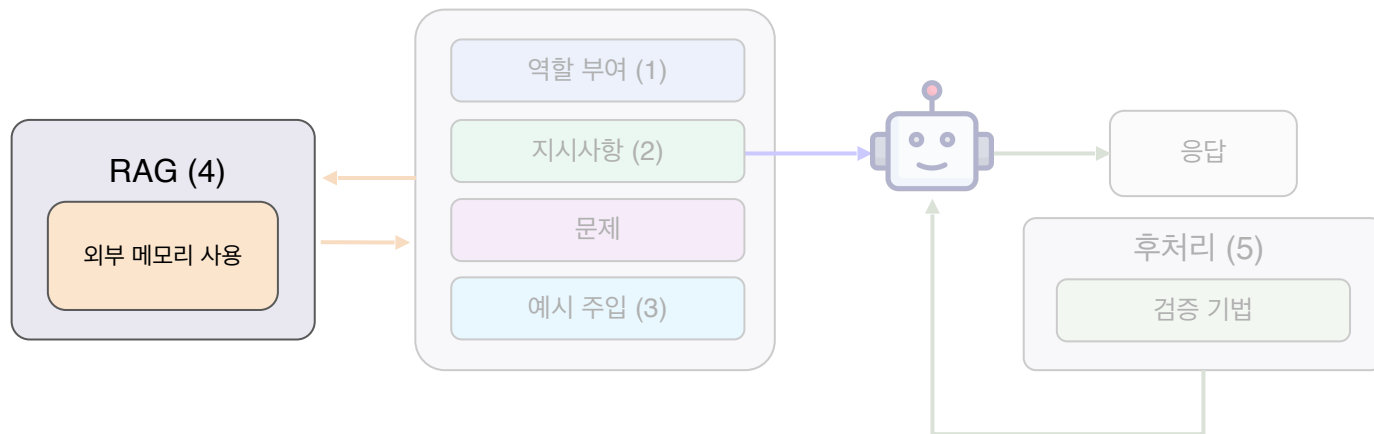
01

RAG의 배경

Parametric vs. Non-parametric Knowledge

Recap

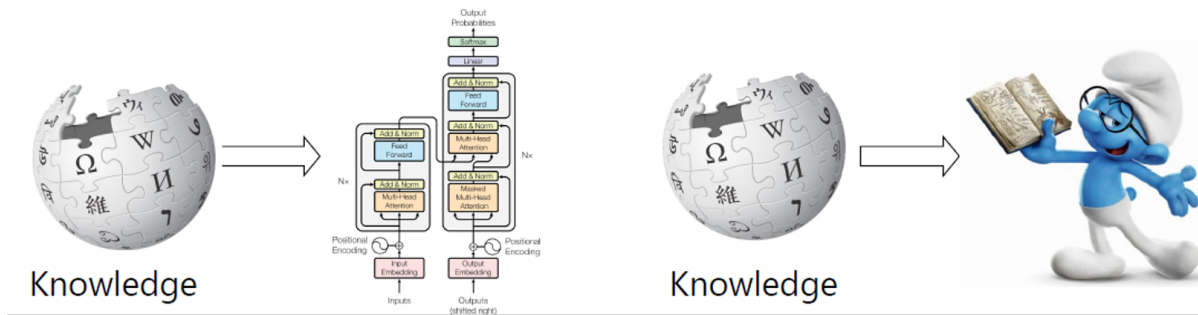
Prompt Engineering 구성



2 Types of Knowledge

분류

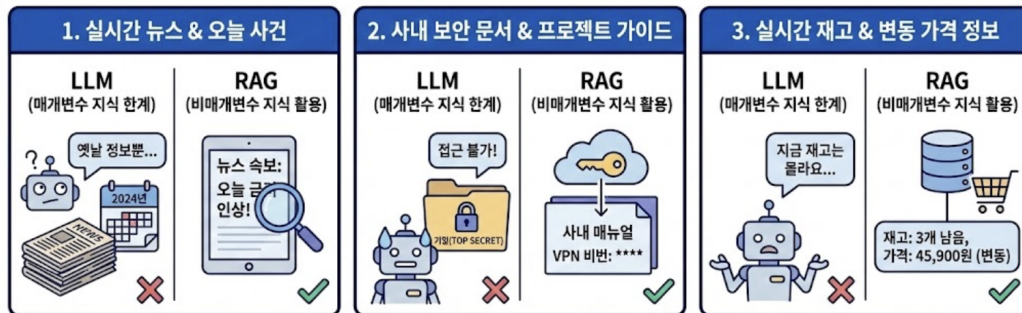
- **Parametric Knowledge:** 학습에 의해 언어 모델 내부에 저장된 지식 (파라미터)
- **Non-parametric Knowledge:** 언어 모델에 없는 외부 지식



Non-parametric Knowledge

외부 지식의 필요성

- 실시간 뉴스 등 최신 정보 검색
- 사내 기밀 문서 이용
- 매 순간 변동하는 재고 & 가격 정보 확인



Outdated Parametric Knowledge

유효기간이 지난 언어 모델의 지식

- **질문:** 현재 미국 대통령은 누구인가?
- **LLM:** Joe Biden입니다. (outdated)
- **External Knowledge** (외부 지식): Donald Trump입니다. (2026 기준)
- 파라미터에 고정된 LLM의 지식은 쉽게 outdated 된다. 하지만 빠르게 업데이트되는 외부 지식을 따라가기 위해 모델을 지속적으로 재학습하는 것은 현실적인 제약이 존재한다.



현재 미국 대통령은 누구인가요?

Joe Biden입니다.



ChatGPT (2022년 학습 완료)

Donald Trump입니다.



(2026년 최신 정보)

유효기간이 지난 언어 모델 지식, 어떻게 해결할까?

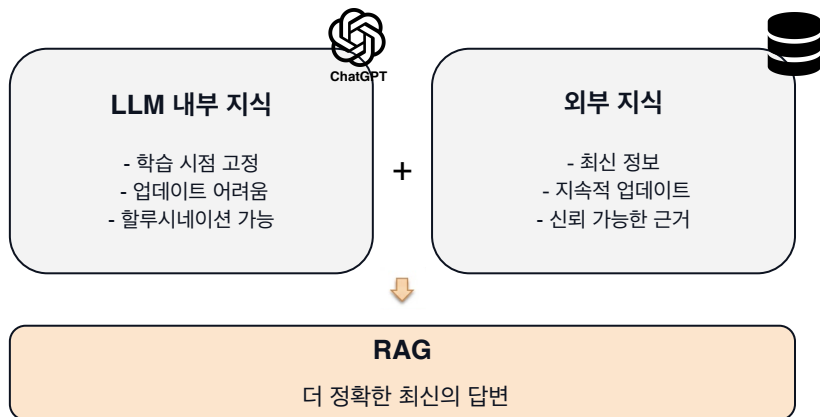
유효기간이 지난 언어 모델의 지식 해결 방법

- 학습 시점 이후의 최신 정보와 도메인 변화로 인해 모델 답변의 정확도와 신뢰도가 떨어짐
- 해결 방법 1: 모델 재학습 또는 재튜닝 - 긴 주기
 - 대규모 데이터 수집과 정제, 학습 비용과 시간이 큼
 - 모델 자체 지식이 갱신되어 일관성이 높음
 - 배포된 모델 전반의 성능을 근본적으로 끌어올림
- 해결 방법 2: Retrieval Augmented Generation - 짧은 주기
 - 외부 지식 저장소를 검색해 근거 문서를 가져오고 그 기반으로 답변 생성
 - 지식 업데이트가 빠르고 비용이 상대적으로 낮음
 - 최신성, 출처성, 근거 기반 응답에 유리함
 - 검색 품질과 문서 품질이 전체 성능을 좌우함

Retrieval Augmented Generation

RAG의 목적

- RAG의 목적은 LLM의 파라미터 기반 내부 지식과 외부 지식 저장소의 정보를 결합하여, 내부 지식만으로는 답변이 어려운 질문에 대해 정확하고 신뢰할 수 있는 응답을 생성하는 것이다.



Retrieval Augmented Generation

RAG의 장점

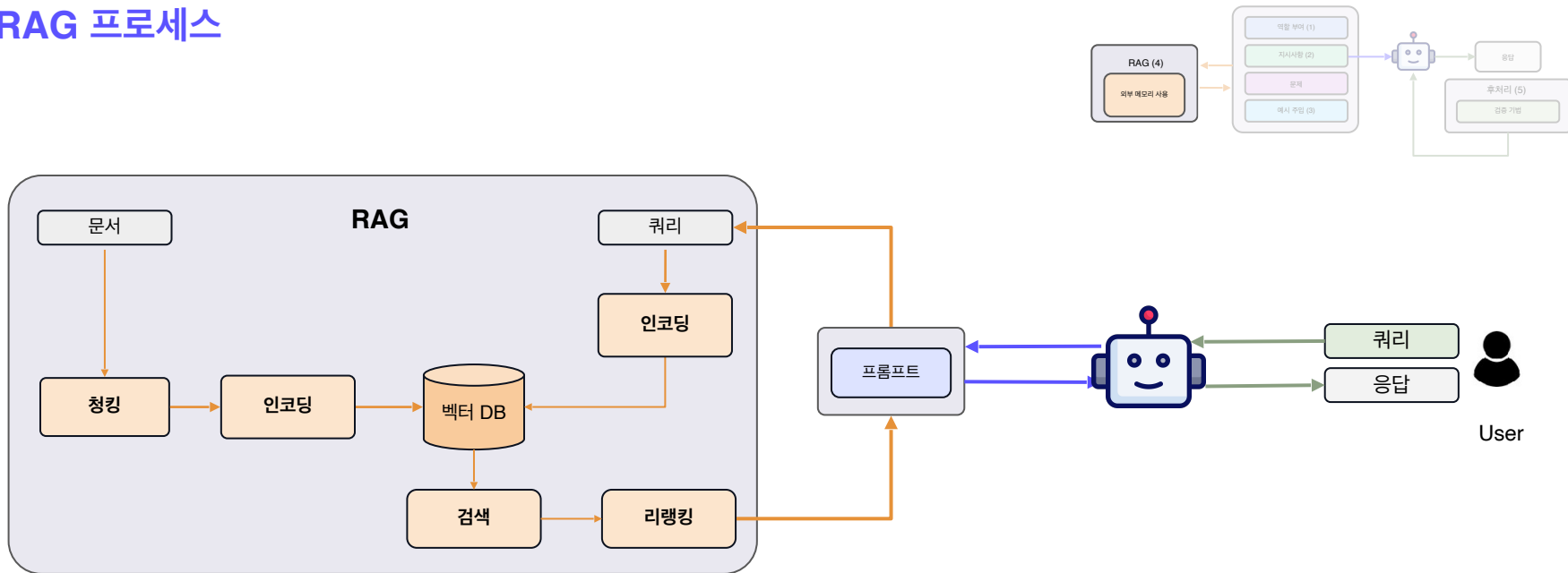
1. 대규모 언어 모델의 재학습(파라미터 업데이트) 없이 외부 지식 활용 가능
2. 외부 지식 참조를 통해 할루시네이션을 감소시키고 정확성을 향상
3. 학습 시점에 고정된 LLM의 한계를 극복하고 최신·도메인 특화 정보 제공

RAG 파이프라인

주요 구성 요소

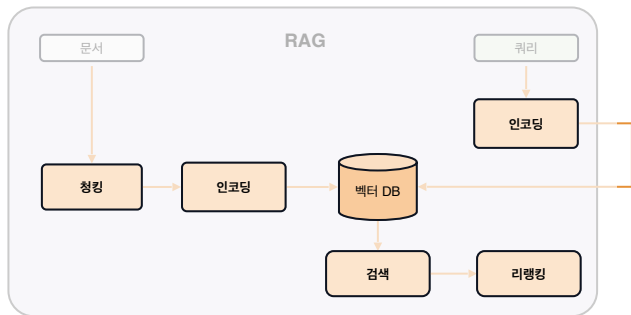
RAG Process Overview

RAG 프로세스



Core Components

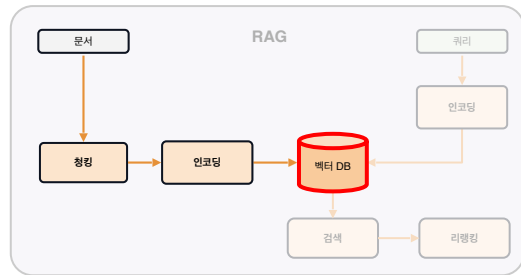
- 지식 베이스 구축(Knowledge Base)
- 청킹 (Chunking)
- 검색 (Retrieval)
- 인코딩 (Encoding)
- ...



Knowledge Base

지식 베이스란?

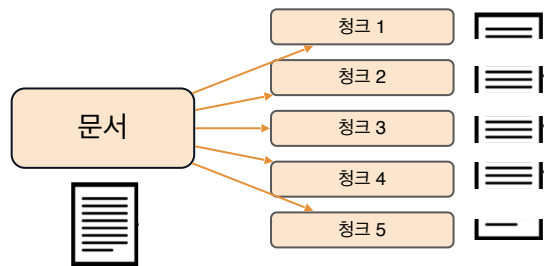
- LLM이 답변을 생성할 때 참조하는 외부 지식 저장소로, 학습 시점에 고정된 파라미터 지식을 보완함
- **지식 베이스 구축 과정**
 - 원본 문서(텍스트, PDF, 도메인 문서 등) 수집 후 문서를 의미 단위로 **칭킹**
 - 각 칭크를 **임베딩 모델로 벡터화**
 - 임베딩을 **벡터 데이터베이스(Vector DB)**에 저장



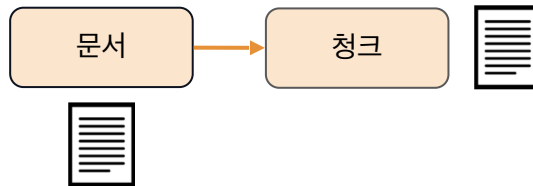
Chunking

청킹이란?

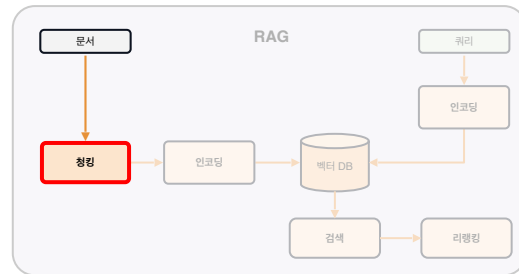
- 하나의 문서를 일정한 토큰 수를 기준으로 더 작은 텍스트 조각(chunk)으로 자르고 조각내서 이것들을 LLM이 검색하고 처리하는 기본 단위로 사용하는 과정
- 단순 청킹 (Fixed-Size Chunking):** 고정된 길이로 데이터를 나누는 것
 - 청킹 단위 (작게 자르는 것 vs. 크게 자르는 것)에 따른 장단점 존재



fine-grained



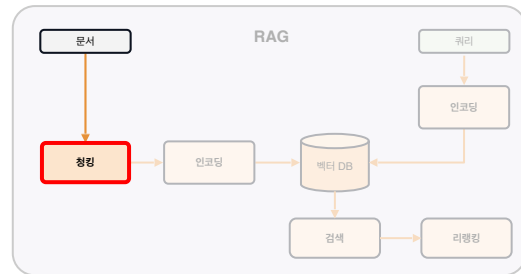
coarse-grained



Chunking

Chunking 단위에 따른 장단점

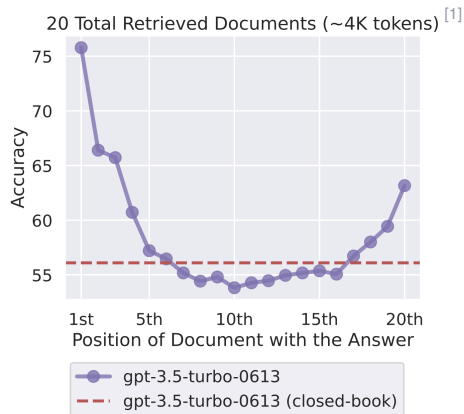
- **Chunking 단위가 작은 경우**
 - 문서가 과도하게 분할되어 chunk 수 증가
 - 검색 대상 증가로 retrieval 시간 증가
 - 임베딩 및 인덱스 크기 증가로 메모리 사용량 증가
 - 문맥이 여러 chunk로 분산되어 의미 단절 발생
- **Chunking 단위가 큰 경우**
 - 하나의 chunk에 다양한 정보가 혼재되어 노이즈 증가
 - 핵심 정보가 문맥 중간에 위치할 경우 **Lost-in-the-Middle** 현상 발생
 - 검색 정확도 및 생성 품질 저하 가능



Chunking

Lost in the middle

- **Chunking** 단위가 큰 경우 **Lost in the middle** 현상 발생: LLM은 긴 컨텍스트의 앞부분(beginning)과 끝부분(end) 정보는 잘 쓰지만, 중간(middle)에 위치한 정보는 잘 활용하지 못함
- 중요한 정보가 문서 중간에 있으면 모델이 놓치거나 무시할 확률이 증가함

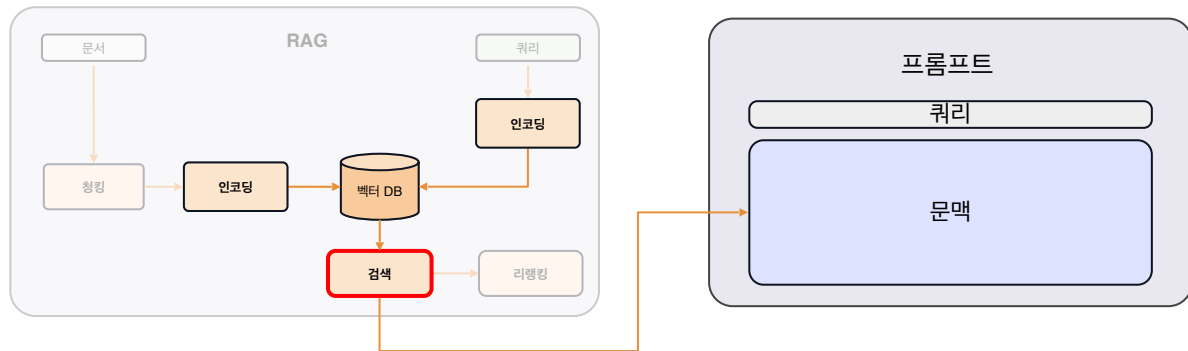


[1] Liu, Nelson F., et al. "Lost in the middle: How language models use long contexts." *Transactions of the Association for Computational Linguistics* 12 (2024)

Retrieval

동작 과정

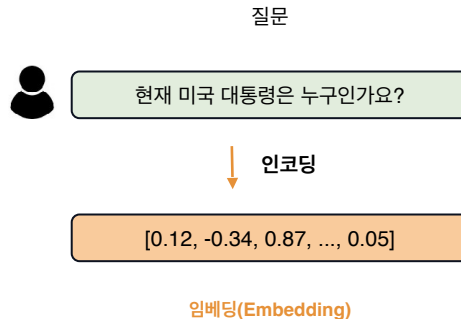
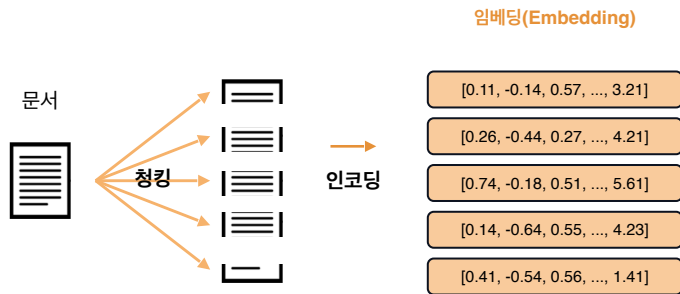
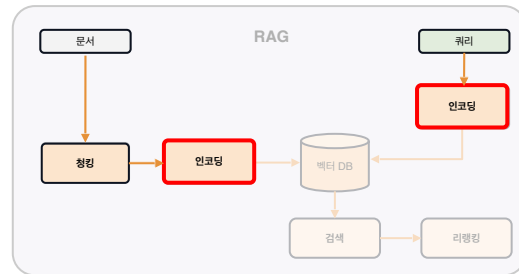
1. 사용자가 질의(쿼리)를 입력
2. RAG 시스템이 **인코딩 모델**을 이용해 사용자 질의를 임베딩 벡터로 변환
3. 질의 벡터와 벡터 DB 내 청크 임베딩벡터들 간의 **유사도 점수**를 계산
4. 질의와의 유사도가 가장 높은 **상위 K개의 청크**를 우선적으로 선택하여 **검색**
5. 이렇게 검색된 청크들을 프롬프트에 확장된 **문맥(Context)**으로 활용



Retrieval

인코딩 과정

- 사람이 읽는 자연어 문장을 컴퓨터가 비교할 수 있는 숫자 표현으로 변환하는 과정
- 질문과 문서 청크를 각각 인코딩하면 임베딩 벡터가 생성됨
- RAG 시스템에서 문서와 질문을 인코딩할 때 사용하는 모델은 동일
- 질문과 문서를 각각 인코딩 후, 두 벡터 간 유사한 정도를 측정하여 알맞은 문서를 검색



Retrieval

Sparse 방식

- 질의와 문서를 희소 벡터로 변환해 동일한 벡터 공간에 매핑, **단어** 기반 유사도를 계산하여 문서를 검색하는 방식
- 검색어에 있는 단어가 많이 나오는 문서를 검색 결과로 제공
- 단어 단위의 중요도를 반영한 벡터를 생성 후 해당 벡터 간의 점수 계산을 통해 문서를 정렬함
- 동일한 단어 벡터 공간에서 질의와 문서 간 **단어** 중첩을 보고 검색을 진행

[Sparse 인코더 예시]

전체 단어 (약 100,000개)

Vocabulary = [apple, banana, car, deep, learning, ..., zookeeper]

100,000 x 1

"Steve Jobs returned to Apple as CEO"

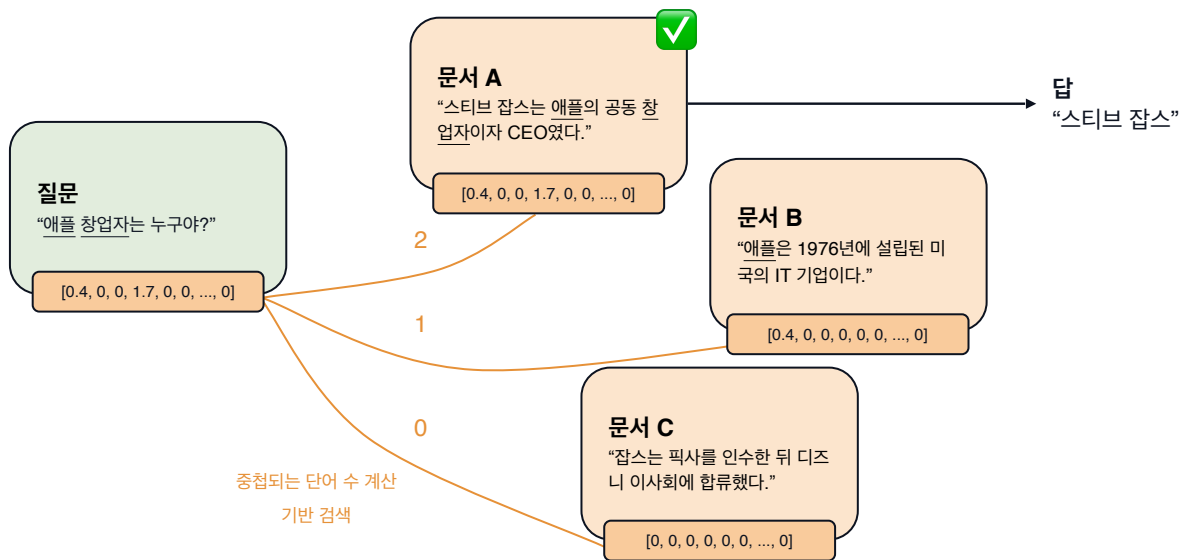
↓ sparse 인코딩

[0.4, 0, 0, 0, 1.7, 0, 0, 0, 0, ..., 0]

100,000 x 1

Retrieval

Sparse 방식 예시



Retrieval

Sparse 방식: BM25

- 단어 빈도와 문서 길이를 함께 고려해 더 정확하게 점수를 매기는 retrieval 방법
 - 쿼리의 단어가 해당 문서에서 등장하는 빈도가 높을수록, 전체 문서에서는 드문 단어일수록 더 높은 점수를 부여함

$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{\text{문서 } D \text{에서 쿼리 단어 } q_i \text{의 등장 빈도} \cdot (k_1 + 1)}{\text{문서 } D \text{에서 쿼리 단어 } q_i \text{의 등장 빈도} + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

문서 길이 정규화
(긴 문서가 무조건 유리해지는 것을 방지)

$$\text{IDF}(q_i) = \ln \left(\frac{\text{전체 문서 수} - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right)$$

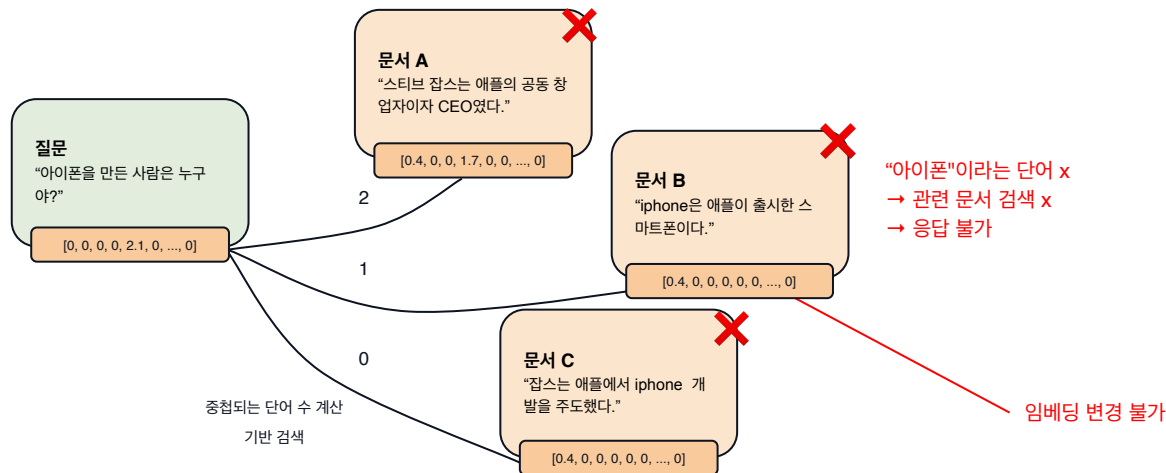
드문 단어일수록 더 중요하게 반영

q_i 를 포함한 문서 수

Retrieval

Sparse 방식의 한계

- Sparse 방식은 중요한 단어들이 정확히 일치해야 하는 경우 성능이 떨어짐
- 하지만 유사하지만 다른 형태의 단어는 벡터 공간에서 다른 단어로 인식 → 질의와 관련된 문서를 찾지 못하는 문제 발생
- 차원의 수가 매우 크고 임베딩 공간 구축된 이후에는 변경 불가



Retrieval

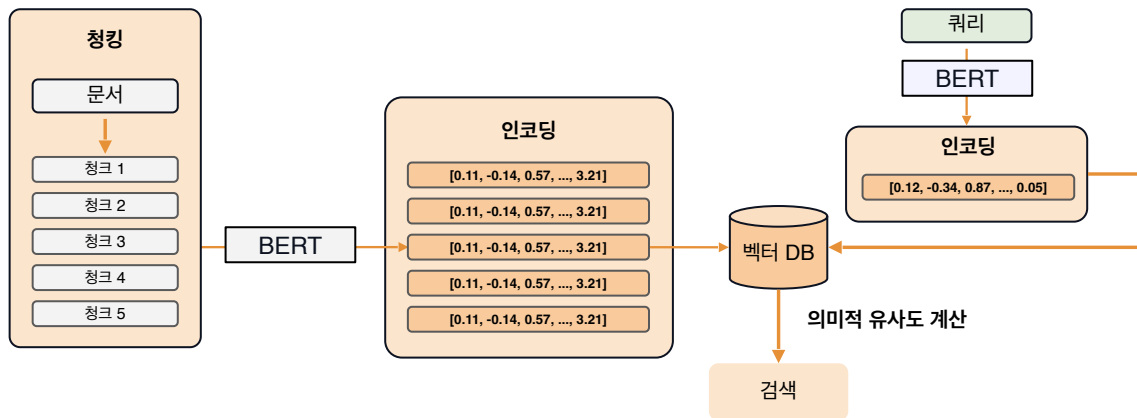
Dense 방식

- 질의와 문서를 동일한 **의미** 기반 임베딩 공간의 **고밀도** 벡터로 압축하여 표현 후, 질의 벡터와 문서 벡터 간 **의미적 유사도**를 계산하여 관련 문서를 검색하는 방식
- 단어의 유사성 또는 맥락을 파악해야 하는 경우 성능이 떨어짐
- Sparse 방식의 단점을 보완
 - 더 작은 차원의 고밀도 벡터 (length=50~1000)
 - 각 차원이 특정 단어에 대응되지 않음, 대부분의 요소 값이 0이 아님

Retrieval

Dense 방식: Dense Passage Retrieval (DPR)

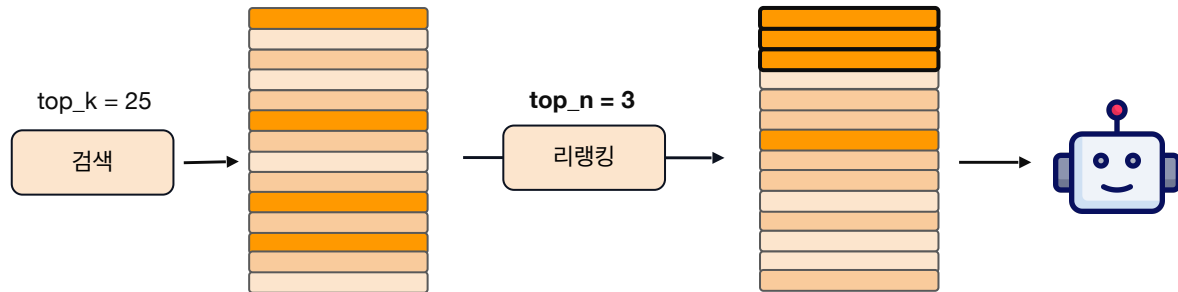
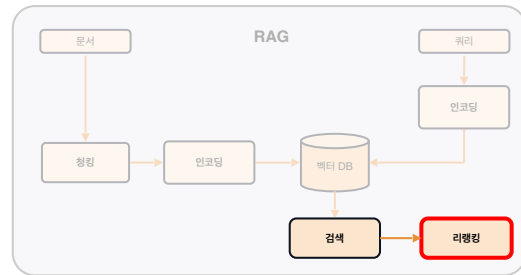
- (Bi-Encoder) 질의와 문서 청크에 대한 임베딩 모델(ex.BERT) 두 가지를 독립적으로 학습함



Reranking

리랭킹이란?

- 여러 모델 (Sparse, Dense 방식)의 검색 결과를 모두 가져와서 상위의 것들을 다시 정렬하는 방법
- 가장 중요한 정보를 **앞쪽에 배치**하여 언어 모델이 더 잘 활용하도록 도움
- “Lost in the middle” 문제 해결



Types of Reranking

유형

1. Rule-based Reranking

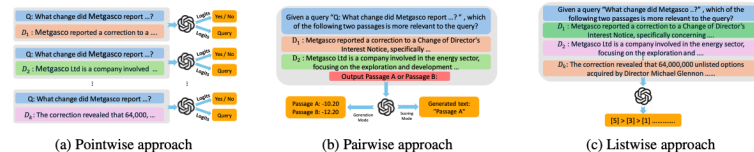
- 사전에 정의된 기준 (relevance, diversity 등)에 따라 랭킹
- 단순하지만 제어가 쉬움

2. Model-based Reranking

- query와 document를 함께 고려하는 LLM 사용
- 보다 복잡한 질문과 문서 간의 관계를 파악할 수 있음
- 더 정확하지만 계산 비용이 큼

Model-based Reranking

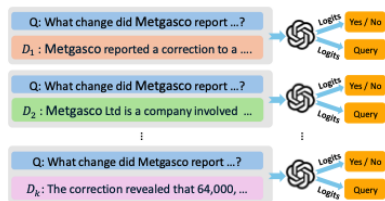
[2]



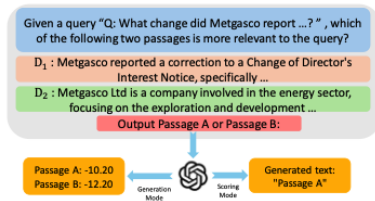
Model-based Reranking

모델 기반 리랭킹의 접근 방식

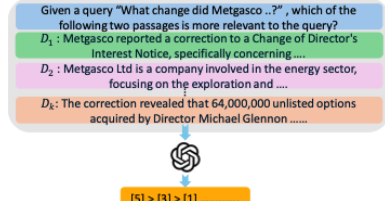
1. **Pointwise approach:** LLM이 각 문서를 하나씩 독립적으로 검토하여 관련성 점수를 매기는 방식
2. **Pairwise approach:** LLM이 두 개의 문서를 서로 비교하여 어느 쪽이 더 우수한지 판단하는 방식
3. **Listwise approach:** LLM이 전체 문서 리스트를 한꺼번에 입력받아 최적의 순서로 재배치하는 방식



(a) Pointwise approach



(b) Pairwise approach



(c) Listwise approach

Summary

RAG Fundamentals

- 2 Types of Knowledge (Parametric vs. Non-parametric)
- RAG의 장점
- RAG 주요 구성요소 (KB, Encoder, Retriever, etc.)
- Lost in the middle과 Chunking
- Sparse vs. Dense 방식
- Reranking 방식



Building intelligence for the future of work